
Pebble AI 项目架构与产品设计报告

Codex

2026-04-20

目录

第一章 执行摘要与阅读指南 5

1.1 项目定义与问题域 5

1.2 核心能力闭环 5

1.3 证据边界与当前定位 6

1.4 阅读建议 7

1.5 本章小结 7

第二章 项目概述与主张 8

2.1 项目定义 8

2.2 目标与非目标 8

2.3 核心价值主张 9

2.4 报告主张与阅读位置 10

2.5 本章小结 11

第三章 背景、冲突与上下文 12

3.1 问题背景 12

3.2 核心冲突 12

3.3 上下文约束 13

3.4 本报告的回答方式 13

3.5 本章小结 14

第四章 术语、符号与前置知识 15

4.1 本章作用 15

4.2 核心术语总表 15

4.3 需要先建立的直觉 16

4.4	读者需要掌握的最小前置知识	17
4.5	本章小结	17
第五章	核心思想与设计原则	18
5.1	本章作用	18
5.2	核心思想：以关系上下文组织连续支持	18
5.3	闭环逻辑：从单次解释走向持续洞察	19
5.4	设计原则	20
5.5	原则如何落到系统面	21
5.6	本章小结	21
5.7	关键设计决策的权衡	21
第六章	方法、建模或关键规则	23
6.1	本章作用	23
6.2	对象建模：Pebble 实际处理的五类核心对象	23
6.3	最小方法闭环	25
6.4	关键规则	26
6.4.1	规则一：所有局部分析都允许显式上下文化	26
6.4.2	规则二：输出必须结构化，而非仅返回自由文本	26
6.4.3	规则三：练习材料是方法闭环中的中间对象	27
6.4.4	规则四：长期洞察以描述性上下文累积，而非一次性定性	27
6.5	方法层的综合理解	28
6.6	本章小结	28
第七章	算法、流程与执行路径	29
7.1	本章作用	29
7.2	总流程概览	29
7.3	路径一：读心翻译的执行链	30
7.4	路径二：模拟陪练的回合流程	31
7.5	路径三：稳态恢复的短流程	32
7.6	横向状态流：关系对象如何跨路径保持连续	32
7.7	执行边界与失败处理	33
7.8	本章小结	33
第八章	系统、数据与资源设计	34

8.1	本章作用	34
8.2	系统分层与边界	34
8.3	前端状态设计	36
8.4	核心模块划分	37
8.5	模块依赖与接口契约	38
8.5.1	模块依赖关系	38
8.5.2	业务层服务接口	38
8.5.3	错误码设计	40
8.6	数据对象与持久化组织	40
8.7	数据库实体关系与 JSONB 结构	41
8.7.1	实体关系	41
8.7.2	JSONB 字段结构	42
8.8	设计系统架构	42
8.8.1	设计令牌	42
8.8.2	组件层级	43
8.8.3	呼吸状态机	43
8.9	匿名会话与登录用户的双轨支撑	44
8.10	数据流实例：从用户输入到持久化	44
8.11	资源与退化策略	46
8.12	本章小结	46
第九章	示例、结果与验证	47
9.1	本章作用	47
9.2	代表性示例	47
9.2.1	示例一：读心翻译的结构化结果	47
9.2.2	示例二：模拟陪练的回合反馈	48
9.2.3	示例三：急救呼吸的状态恢复反馈	48
9.3	当前已观察结果	48
9.4	验证证据	49
9.5	尚未完成的验证边界	50
9.6	本章小结	51
第十章	项目进度	52
10.1	本章作用	52

10.2 已完成的能力面	52
10.2.1 前端交互层	52
10.2.2 后端服务层	53
10.2.3 数据持久化层	54
10.2.4 测试覆盖	54
10.3 技术实现状态	55
10.3.1 设计系统落地	55
10.3.2 OpenSpec 变更归档	55
10.4 当前阻塞与待完善项	56
10.4.1 后端 BFF 剩余工作	56
10.4.2 移动端与可访问性	56
10.4.3 功能缺口	56
10.5 里程碑回顾	56
10.6 本章小结	57
第十一章 风险、局限与下一步交付	58
11.1 本章作用	58
11.2 主要风险	58
11.2.1 技术风险	58
11.2.2 产品风险	59
11.2.3 合规与伦理风险	60
11.3 已知局限	60
11.4 交付边界	60
11.5 下一步建议	61
11.6 本章小结	62

第一章 执行摘要与阅读指南

1.1 项目定义与问题域

本章用于给出整份报告的总导读。本文将 Pebble 定义为一个以关系上下文为核心的 AI 情绪防御与沟通辅助系统。它面向处在职场、家庭或亲密关系压力中的用户，目标是在高压互动发生时提供可持续的理解、练习与自我调节支持，而非只输出一一次性情绪标签或单轮回复建议。

从问题域看，Pebble 关注的是同一句话在不同关系结构、互动历史与当前情境中会呈现出不同的风险含义，因此系统需要同时处理语言表层、关系背景、应对策略与用户状态恢复四类信息。基于这一判断，Pebble 的产品主线不应被理解为单点问答工具，而应被理解为围绕复杂人际互动展开的连续支持链路。

1.2 核心能力闭环

Pebble 当前适合采用“四段闭环”的方式概括其能力结构。

- 对话解码：用户输入一句令自己不适、困惑或难以回应的话语，系统输出表层语义、潜台词与候选回复方案。
- 关系陪练：用户在场景化对话中持续练习回应方式，并从系统获得即时反馈与策略提示。
- 呼吸稳态：用户在情绪负荷过高时进入短时稳定化流程，以降低后续判断和表达的失真风险。
- 关系档案与长期洞察：系统持续记录关系对象、互动背景和练习结果，使分析与建议能够逐步积累上下文。

为了让读者在进入后文章节前先建立稳定的全局心智模型，图 1.1 将 Pebble 的导读层压缩为一张总览图。该图强调 Pebble 的价值来自四段能力的闭环，而非某一个单点模块的孤立表现。

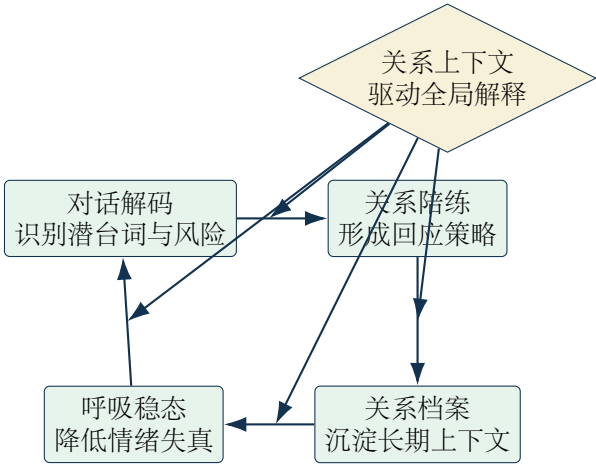


图 1.1: Pebble 总览闭环，突出关系上下文如何统摄解码、陪练、稳态恢复与长期记忆。

这一闭环意味着 Pebble 的核心价值来自能力之间的联动。解码模块负责帮助用户理解当下对话；陪练模块负责把理解转化为可重复训练的表达能力；呼吸模块负责在高压时刻提供最小可用的生理稳定支持；关系档案负责把一次次局部互动沉淀为长期记忆，使系统能够在后续分析中引入更细粒度的情境差异。

1.3 证据边界与当前定位

本报告后续章节会同时使用两类信息来源。

- design intent：用于说明 Pebble 试图建立什么样的产品结构、能力边界与用户体验。
- 已观察事实：用于说明哪些系统能力已经得到直接观察。

基于这一区分，本文后续论述会把系统愿景、已验证能力与交付边界明确拆开，以保证判断强度与证据强度保持一致。

与此同时，第一章不承担展开实现细节的职责。接口字段、认证路径、关系数据模型、测试矩阵、迁移历史与当前阻塞等内容会在后续章节分别展开，其中与真实实现状态直接相关的信息默认集中进入“项目进度”章节。本章只保留理解整份报告所必需的最小事实，以

避免导读层被工程细节淹没。

1.4 阅读建议

如果读者首先关心 Pebble 要解决什么问题，以及它为何需要围绕关系上下文组织能力，建议优先阅读第二章与第三章。第二章会明确项目主张、目标边界与读者应如何理解 Pebble 的系统使命；第三章会展开背景、冲突与上下文，说明这一类产品为何不能仅依赖单句文本分析。

如果读者首先关心系统如何工作，建议继续阅读第四章至第八章。第四章负责定义关键术语和前置概念，第五章概括核心设计原则，第六章和第七章分别展开流程与系统资源设计，第八章说明示例、验证与可接受结果。若读者关心当前真实进展、里程碑与风险，则应直接阅读第九章和第十章。

读完整份报告后，读者应形成以下三个稳定判断。

- Pebble 的系统边界建立在“关系上下文驱动的沟通支持”之上。
- Pebble 的产品能力应以解码、练习、稳态恢复与长期记忆的联动来理解。
- Pebble 已经具备进入架构与产品设计讨论的系统骨架，但实现成熟度、验证深度与交付边界仍需要在后文按证据层逐项区分。

1.5 本章小结

本节读完后，读者应能复述 Pebble 的四段能力闭环（对话解码、关系陪练、呼吸稳态、关系档案），并指出关系上下文在整个闭环中的中心作用。

第二章 项目概述与主张

2.1 项目定义

Pebble 是一个面向复杂人际互动场景的 AI 情绪防御与沟通辅助系统。本文对该项目的核心定义是：系统以关系上下文为解释中心，把对话解码、关系陪练、稳态恢复与关系档案管理组织为同一条用户支持链路，使用户能够在高压沟通中先理解局势，再选择应对方式，并在后续互动中持续积累策略经验。

这一项目定义同时包含两个层次。

- Pebble 是产品对象，关注用户在真实关系中的表达压力、误读风险与边界维护需求。
- Pebble 是系统对象，要求前端交互、分析服务、关系数据与反馈机制能够围绕同一组关系上下文协同工作。

后续章节会分别展开这两个层次，但本章先把项目主张收束为统一的问题陈述。

2.2 目标与非目标

Pebble 的直接目标有三项。

- 帮助用户更准确地解释复杂对话中的潜台词、情绪信号与关系风险。
- 帮助用户把一次性的理解结果转化为可练习、可比较、可回顾的回应能力。
- 帮助用户在高压时刻获得最小可用的稳定支持，并把关系背景纳入后续分析，使系统逐步形成更细粒度的策略适配能力。

本项目的非目标同样需要明确。Pebble 不以心理诊断、医学治疗或人格定性为目标，也不

尝试替代用户在真实关系中的判断责任。系统可以提供分析、建议与练习环境，但最终如何理解对方、是否采纳建议、是否调整关系边界，仍由用户自行决定。后续涉及画像、洞察或策略提示的章节，都应在这一边界内理解其作用。

2.3 核心价值主张

Pebble 的核心价值不在于把单句文本送入模型后返回一段解释，而在于把“关系背景”持续纳入分析过程。对于同一句话，来自母亲、伴侣、同事或上级的表达，往往意味着不同的互动历史、风险结构与回应策略。若系统忽略关系上下文，就只能输出抽象而平均化的建议；若系统能够维护关系档案与长期观察，它就有条件在解释与建议中引入更贴近用户现实处境的差异化判断。

这一价值主张进一步带来两个产品层面的意义。

- Pebble 可以从一次性分析工具演进为持续性关系洞察系统。
- Pebble 可以把“理解”与“应对”连接起来，使用户不只得到一个分析结果，还能进入陪练、记录、复盘与下次适配的循环。

这一循环决定了项目后续架构、数据组织和页面设计的基本方向。

表 2.1 对本章的项目主张做进一步压缩。它的作用是把 Pebble 的系统对象、直接目标与边界约束放在同一张表中，便于读者在进入背景章之前先形成稳定的比较框架。

表 2.1: Pebble 项目主张概览，突出系统对象、目标与边界的对应关系。

维度	Pebble 的主张	本章给出的边界
系统对象	关系上下文驱动的 AI 情绪防御与沟通辅助系统	同时覆盖产品体验、分析服务与关系数据协同
直接目标	帮助用户理解高压对话、训练回应能力，并在需要时快速稳定状态	不把单次情绪识别视为最终目标，而强调连续支持链路
差异化来源	关系档案与长期观察可让同一句话获得更贴近情境的解释与建议	不承诺脱离关系背景即可给出高质量判断
非目标	系统提供分析、建议与练习环境	不承担心理诊断、医学治疗或替代用户判断的职责
报告定位	以后续章节展开产品结构、背景、系统设计与验证路径	当前章只立题，不展开接口、迁移历史和工程阻塞

2.4 报告主张与阅读位置

本报告对 Pebble 的主张可以概括为三点。

- Pebble 应被理解为关系上下文驱动的沟通支持系统，而非孤立的情绪识别工具。
- Pebble 的能力结构必须以解码、练习、稳态恢复与长期记忆的联动来描述，缺失其中任一环节都会削弱系统闭环。
- Pebble 的产品价值来自用户问题、关系数据与交互流程之间的协同设计，因此后续分析不能只停留在页面视觉或接口列表层面。

从全篇结构看，本章承担“立题”功能。第三章会进一步解释这一项目为何成立，并把背景、冲突与上下文展开为更完整的问题域分析；第四章至第八章会把术语、原则、流程、系统设计与验证路径依次展开；第九章与第十章则分别收束当前进展与风险边界。读者完成本章后，应已理解 Pebble 关注什么问题、其系统边界为何成立，以及后续各章将如何围绕这一主张逐步展开。

2.5 本章小结

本节读完后，读者应能说明 Pebble 的直接目标与非目标边界，并解释为什么系统价值不能只用单句情绪识别来理解。

第三章 背景、冲突与上下文

3.1 问题背景

Pebble 所面对的问题首先来自高压人际互动的普遍性。用户在职场、亲密关系和日常社交中，常常会接触到带有指责、控制、道德绑架或情绪投射色彩的话语。这类话语在表面上可能只是催促、抱怨或询问，但在具体关系中往往还携带更复杂的压力结构。用户若只能凭即时情绪反应作答，回应质量就容易受到恐惧、内疚、愤怒或自我怀疑的影响。

在这一背景下，Pebble 试图处理的并非抽象的文本理解任务，而是一个更贴近真实沟通场景的问题：当用户收到一句令人不适的话时，系统能否帮助其先分辨真实含义，再形成较为稳健的回应，并在后续互动中持续积累更适配的策略。读心翻译器、模拟陪练场和急救呼吸三个功能原型都围绕这一目标展开，只是它们分别切入了理解、练习与稳定状态的不同阶段。

3.2 核心冲突

本项目的核心冲突在于，用户需要的是带关系上下文的沟通支持，而多数传统交互系统提供的是脱离上下文的单轮回答。若系统只处理当前这句话，它通常能够生成语义层面的解释，却难以判断这句话在特定关系中究竟意味着催促、试探、操控、情绪投射还是边界侵犯。对于 Pebble 这类产品而言，真正有价值的判断并不来自单句文本本身，而来自文本、关系历史、场景压力与用户目标之间的联动。

这一冲突进一步表现为两个层面的张力。

- 用户希望快速获得帮助，因此系统需要在高压时刻给出低摩擦、可执行的支持。
- 用户又希望系统足够懂自己所处的关系背景，因此系统不能长期停留在一次性分析层面。

前者要求产品具备即时性，后者要求系统具备持续记忆与情境适配能力。Pebble 的架构与产品设计正是在这两类要求之间寻找可行平衡。

3.3 上下文约束

Pebble 所处的上下文并不只是“做一个 AI 页面”。从产品侧看，系统必须同时处理至少三类不同但彼此相关的需求。

- 即时解释：用户在收到令人不适的话语后，希望快速获得潜台词分析与候选回复。
- 安全练习：用户希望在真实冲突发生前进行模拟对话，观察自己是否仍然陷入辩解、讨好或情绪升级。
- 状态恢复：在用户已被情绪淹没时，系统需要提供比分析更基础的稳态支持。

从系统侧看，Pebble 还面临两类结构性约束。

- 关系上下文必须可持续维护，否则同一句话在不同关系中的差异无法被稳定表达。
- 系统边界必须保持克制，否则产品很容易滑向心理诊断、人格贴标签或过度承诺的方向。

也正因如此，Pebble 在相关设计文档中强调去标签化、渐进累积、用户主权与隐私优先。这些约束决定了本报告后续不能把产品简单写成“更聪明的情绪识别器”，而必须把它写成一个围绕关系情境组织支持链路的系统。

3.4 本报告的回答方式

针对上述背景与冲突，本报告给出的回答方式是把 Pebble 解释为一个关系上下文驱动的闭环系统。第一章和第二章已经分别确立了项目定义与主张；从本章开始，报告将进一步说明这一主张为何成立。具体来说，后续章节会依次回答四个问题：为什么单句分析不足以支撑高质量沟通支持，Pebble 需要哪些核心概念与设计原则，系统如何把解码、陪练、稳态恢复与长期记忆组织成可协同的结构，以及当前产品骨架与实现边界处在什么位置。

因此，本章的作用是把读者带到一个更准确的问题入口，而不提前展开最终设计细节。后文结构建立在以下判断之上：Pebble 所讨论的是如何在复杂关系中为用户建立一个可持续的理解、练习与应对支撑系统。

3.5 本章小结

本节读完后，读者应能描述 Pebble 面对的核心冲突——用户需要带关系上下文的沟通支持，而多数系统提供的是脱离上下文的单轮回答——并指出这一冲突在产品层面的两个表现。

表 3.1 把上述需求与约束压缩为对照格式，便于读者在进入后续章节前快速建立比较框架。

表 3.1: Pebble 的需求类型与对应的系统约束。

需求类型	用户侧表现	系统侧约束
即时解释	收到不适话语后快速获得潜台词分析	低延迟响应；允许关系上下文进入主路径
安全练习	在真实冲突前模拟对话并观察自身反应	回合状态机必须稳定；场景切换与重启必须可预测
状态恢复	情绪过载时获得最小可用的稳定支持	不依赖后端；支持暂停与可控节奏
长期洞察	多轮互动后获得更贴近情境的建议	关系档案必须可持续维护；去标签化表达

第四章 术语、符号与前置知识

4.1 本章作用

本章用于建立后续章节共享的术语框架。Pebble 的讨论对象横跨产品体验、关系数据、交互反馈与分析服务，如果读者在进入架构与方法章节前没有先区分这些对象，就容易把系统的设计目标、已观察能力与用户感知层面的体验描述混写在一起。因此，本章先给出后文最常出现的概念，并说明它们在本文中的含义边界。

对于第一次接触 Pebble 的读者，建议把本章理解为一份最小词汇表。后文若出现相同术语，默认沿用本章定义；若某一章节对术语做了更细的展开，应视为在本章定义基础上的局部细化，而不是重新定义对象。

4.2 核心术语总表

表 4.1 汇总了后文最重要的术语。它的目的在于确保读者在阅读产品主张、系统设计和验证部分时，对同一个词保持同一种理解，而不在这一章提前展开全部实现细节。

表 4.1: Pebble 核心术语与本文中的使用边界。

术语	本文中的含义	阅读时应注意的边界
关系上下文	与某个关系对象相关的背景信息、互动历史、预期目标与场景补充	它是 Pebble 解释对话与生成建议的中心条件，不应被理解为附属备注
对话解码	对当前输入话语做表层语义、潜台词与回应候选的分析过程	它解决“先理解”问题，不单独承担长期训练与状态恢复职责
灰岩回复	尽量降低情绪刺激、减少解释与对抗升级的回应策略	它是策略风格，不等于冷漠或机械回避一切沟通
模拟陪练场	通过场景化对话练习回应方式，并获得即时反馈的交互环境	它强调训练与复盘，不直接替代真实关系中的最终决策
急救呼吸	在情绪负荷过高时提供短时稳态支持的生理调节模块	它处理的是状态恢复，不负责完成语言分析本身
关系档案	面向具体关系对象持续维护的结构化信息集合	它既服务即时分析，也为长期洞察和策略适配提供基础
已观察事实	已经在当前工作区、页面或接口中得到直接观察的信息	该层只描述真实存在的能力，不自动外推为未来保证
design intent	报告中关于目标结构、产品方向与系统设计的规划性描述	该层表达的是要建成什么，不等于当前已经全部实现

4.3 需要先建立的直觉

在 Pebble 的语境中，读者至少需要建立三个直觉判断。

- 语言分析不是孤立任务。同一句话在不同关系中可能意味着完全不同的压力结构，因此 Pebble 不能只围绕文本表面做响应。
- 高质量支持不等于一次回答。用户往往需要先理解，再练习，再恢复状态，并在后续互动中继续积累上下文。
- 系统价值来自协同，而非单模块堆叠。解码、陪练、稳态恢复与关系档案只有在同一条支持链路中协同工作时，Pebble 的整体主张才成立。

这三个直觉决定了后文的阅读方式。若读者始终把 Pebble 理解成“一个分析文本的页面”，那么第五章之后的很多设计选择会显得过度复杂；若读者接受 Pebble 是一个围绕复杂关系

组织支持链路的系统，那么这些设计选择就会表现为必要的结构条件。

4.4 读者需要掌握的最小前置知识

理解本报告并不要求读者具备心理学、临床医学或复杂机器学习背景，但仍需掌握若干最小前提。

- 读者应理解“上下文会改变同一句话的解释结果”这一基本事实。
- 读者应接受产品设计中的边界意识，即系统可以提供支持，但不替代用户做价值判断。
- 读者应能区分界面层、服务层与数据层这三类对象，避免把页面文案、分析逻辑和关系数据视为同一种设计元素。
- 读者应能区分规划性描述与已验证能力，这一点对理解本报告中的 design intent 与已观察事实尤为重要。

如果读者具备以上前置知识，后续章节中的系统边界、交互流程和验证策略就可以被直接理解；若缺少这些前提，则建议在继续阅读前先回看第一章与第二章，重新确认 Pebble 的问题域、目标边界与核心主张。

4.5 本章小结

本节读完后，读者应能定位”关系上下文”、”对话解码”、”关系档案”三个术语的首次定义位置，并说明它们之间的依赖关系。

此外，本章对术语做了产品层与技术层的分层说明：”读心翻译”是产品功能名称，其技术实现对应”对话解码”过程；”模拟陪练场”是产品功能名称，其技术实现对应”关系陪练”过程。

第五章 核心思想与设计原则

5.1 本章作用

前四章已经分别说明了 Pebble 的问题域、项目主张、背景冲突与术语边界。本章进一步回答一个更基础的问题：为什么 Pebble 必须围绕关系上下文组织为一条连续支持链路，以及哪些原则在后续方法、流程与系统设计中不应被破坏。

这一章的任务是给出一套足以约束后续设计判断的核心思想。缺少这一层时，后文关于解码、陪练、稳态恢复、关系档案与洞察累积的设计容易被误读为若干分散功能的叠加；核心思想被固定后，后续章节中的模块划分、数据组织和交互路径就会呈现为围绕同一目标展开的结构安排。

5.2 核心思想：以关系上下文组织连续支持

Pebble 的核心思想可以概括为一句话：系统围绕具体关系对象维护可持续更新的上下文，并把这一上下文注入用户每一次理解、练习、恢复与复盘过程，使支持能力随着互动历史逐步变得更贴近真实处境。

这一思想包含四个相互依赖的判断。

- 对复杂对话的解释不能脱离关系背景。同一句表达在亲密关系、家庭关系或职场关系中往往具有完全不同的压力结构、风险信号与回应空间。
- 用户需求体现为连续支持。用户需要先理解话语，再决定如何回应，在情绪负荷过高时恢复状态，并在后续互动中持续校正判断。
- 关系洞察应以渐进累积方式形成。系统对关系对象的理解来自多轮互动、场景补充与重复出现的行为线索，而不是一次输入后的永久结论。

- 产品价值来自闭环，而不是单点准确率。即使某一次分析文本的结果看似合理，若无法被带入陪练、记录和后续适配，系统仍难以形成稳定的长期价值。

因此，Pebble 的“核心思想”并非某一种单独的模型能力，也不是一条孤立的界面流程。它是一种组织原则：把关系对象作为解释中心，把用户支持链路作为设计中心，把长期累积作为系统演化中心。

5.3 闭环逻辑：从单次解释走向持续洞察

在这一思想下，Pebble 的能力结构可被理解为一个由四段组成的闭环。第一段负责理解当前输入，第二段负责把理解结果转化为可练习的应对方案，第三段负责在用户状态失稳时提供最小可用的恢复支撑，第四段负责把新一轮互动中的观察沉淀回关系档案，为后续解释提供更稳定的上下文条件。

这一闭环的关键不在于模块数量，而在于信息如何回流。系统若只停留在“输入一句话，返回一段解释”的层面，就只能生成局部答案；系统若能把关系背景、观察记录和用户选择持续带回下一轮分析，才有条件形成逐步增强的情境适配能力。由此，Pebble 的长期价值不应只用单次回答的表面可读性来衡量，还应考察它是否使用户在多轮互动中逐渐建立更稳的判断框架和边界策略。

从报告结构看，这一闭环同时解释了后续章节的组织方式。

- 方法与建模章节会说明系统如何把关系对象、对话输入与观察记录组织成可被持续调用的分析条件。
- 流程与执行路径章节会说明用户如何在页面之间完成理解、练习、恢复与回流。
- 系统与数据设计章节会说明哪些数据必须长期维护，哪些状态只在局部交互中存在。
- 示例、结果与验证章节会说明系统价值应如何通过闭环效果而非单点表现来理解。

5.4 设计原则

核心思想若要在实现中保持稳定，至少需要六条设计原则作为约束。它们并非文案层面的偏好，而是后续架构与交互设计的判断准则。

表 5.1 汇总了上述原则及其约束含义。

表 5.1: Pebble 核心设计原则及其在系统中的约束含义。

原则	含义	对后续设计的约束
关系上下文优先	解释与建议应围绕具体关系对象组织，而非只依据输入文本表面语义	后续分析服务、状态管理与页面流程都应允许关系信息进入主路径
先理解后应对	系统先帮助用户辨识语气、潜台词与风险，再进入回应建议与练习阶段	解码输出应承担中介角色，不能跳过理解层直接给模板回复
渐进累积	关系洞察来自多轮互动中的重复信号与上下文补充	数据层应支持持续更新和回看，避免把一次分析写成永久性定论
去标签化	系统用观察性、描述性语言表述模式，不对关系对象做武断定性	洞察表达应强调行为线索与证据，不把诊断性词汇当成默认输出
用户主权	用户对关系档案、洞察记录与策略采纳保持最终决定权	交互设计应保留查看、编辑、忽略或删除的空间，避免系统替用户下判断
边界克制与隐私优先	系统只收集完成支持所需的最小信息，并对敏感关系信息保持克制处理	后续设计应避免不必要的数据扩张，并明确区分即时分析与长期沉淀的边界

上述原则可以进一步压缩为三组更高层的约束关系。

- 以关系上下文优先、先理解后应对为代表的原则，约束系统应如何组织分析入口。
- 以渐进累积、去标签化为代表的原则，约束系统应如何形成和表达关系洞察。
- 以用户主权、边界克制与隐私优先为代表的原则，约束系统在提供支持时不得越过的责任边界。

5.5 原则如何落到系统面

这些原则的意义在于，它们能够直接解释若干后续设计选择为何成立。

- 若坚持关系上下文优先，那么“当前关系对象”的选择与调用就不能只是页面附属信息，而应成为解码与建议路径中的主条件。
- 若坚持先理解后应对，那么系统输出就应先呈现表层语义、潜台词与情绪风险，再进入回复建议与练习动作。
- 若坚持渐进累积与去标签化，那么关系档案中沉淀的内容就应以观察记录、证据线索与可更新摘要为中心，而非一次性贴上结论性标签。
- 若坚持用户主权与边界克制，那么 Pebble 的角色就应被限定为支持性工具，系统可以提供洞察、建议与训练环境，但不能越过用户的价值判断与现实决策责任。

由此可见，第五章提出的是一组能够约束交互流程、服务编排、数据模型与内容表达的基础规则。后续章节若出现任何局部设计选择，读者都可以回到本章检验一个问题：该选择是否仍然服务于“关系上下文驱动连续支持”这一核心思想，并且没有破坏本章确立的设计原则。

5.6 本章小结

本节读完后，读者应能解释为什么 Pebble 必须围绕关系上下文组织为连续支持链路，并列举至少三条约束后续设计选择的核心原则。

5.7 关键设计决策的权衡

Pebble 的若干核心设计并非唯一可行路径。本节明确三项关键决策的主要权衡，以及替代方案被排除的主要理由。

表 5.2 对上述三项决策做了进一步压缩。

表 5.2: Pebble 核心设计决策的权衡对照。

决策	选定方案	主要收益	主要代价与被排除的替代方案
分析入口的组织方式	关系上下文优先：以关系对象为中心组织分析	同一句话在不同关系中获得差异化解释；长期洞察可渐进累积	需维护额外数据层；被排除的替代是纯文本单轮分析——更简单即时，但失去情境适配与长期记忆能力，与产品核心价值主张冲突
系统架构深度	Next.js BFF + PostgreSQL + Drizzle 六层全栈结构	支持数据沉淀、匿名/登录双轨、练习本持久化；确定性 fallback 可在服务端执行	引入数据库运维、迁移管理复杂度；被排除的替代是纯前端直接调用 LLM API——上线更快，但失去用户数据持久化、跨设备同步与精细权限控制
用户身份策略	Cookie-based guest session + 可选 Supabase Auth 登录	降低首次体验门槛；匿名用户可先体验核心功能	数据模型需同时支持 guestSessionId 和 userId；被排除的替代是强制登录——数据模型更简单，但提高使用门槛，与”低摩擦支持”的产品目标冲突

第六章 方法、建模或关键规则

6.1 本章作用

前一章给出了 Pebble 的核心思想与设计原则，本章进一步回答“系统如何把这些原则落成可执行的方法结构”。这里的“方法”主要指对象如何被建模、输入如何被解释、结果如何沉淀，以及不同功能模块之间如何组成最小闭环。它既要足够抽象，能够解释全局设计；也要足够贴近系统对外呈现的输入、输出与数据边界，避免停留在口号层。

因此，本章不把 Pebble 写成一个单独的模型调用接口，而是把它写成一个由关系对象、当前输入、分析结果、训练记录与长期洞察共同构成的方法系统。后续流程章与系统章会分别展开执行路径与工程结构；本章先固定其方法对象、作用关系与关键规则。

6.2 对象建模：Pebble 实际处理的五类核心对象

Pebble 当前的方法结构可以压缩为五类核心对象。它们对应的并非抽象概念，而是当前工作区中已经可观察到的服务输入、输出与持久化载体。

表 6.1 对上述五类对象做了汇总。

表 6.1: Pebble 方法层的核心对象及其在系统中的作用。

对象	在方法层中的定义	当前系统中的典型承载位置
关系对象	被持续维护的上下文单元，包含关系类型、对方特点、期望结果、情境补充与生成上下文	关系服务与关系节点数据
当前输入	用户此刻需要理解或回应的文本、消息或对话轮次	解码请求文本、模拟陪练中的当前回合消息
分析结果	系统对当前输入形成的结构化解释，包括潜台词、风险类型、情绪得分与回应候选	解码服务输出、陪练侧栏反馈
练习记录	用户从分析或模拟中选取、保存并准备回看的应对内容	练习本条目与收藏结果
长期洞察	围绕具体关系逐步累积的观察摘要、生成上下文与策略适配条件	关系档案中的结构化字段与生成上下文

这五类对象之间的关系决定了 Pebble 的方法边界。

- 关系对象负责提供情境解释条件。
- 当前输入负责触发一次局部分析或一轮练习互动。
- 分析结果负责把原始输入转换成用户可理解、可比较、可操作的结构。
- 练习记录负责把一次性的建议转化为可积累的经验材料。
- 长期洞察负责把多轮互动中的局部观察重新汇回关系对象，形成下一次分析的先验条件。

6.3 最小方法闭环

在对象建模明确之后，Pebble 的方法可以被概括为一条四步闭环：条件注入、局部分析、反馈转化、长期沉淀。图 6.1 对这一闭环做了压缩表达。

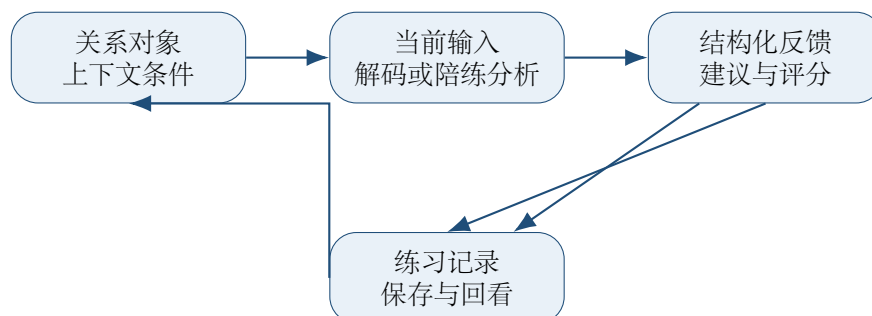


图 6.1: Pebble 的最小方法闭环：关系上下文进入当前分析，分析结果再回流为练习材料与长期洞察条件。

这条闭环中的每一步都有明确的方法含义。

- 条件注入：系统先收集与当前关系相关的最小必要上下文，使分析不只依赖孤立文本。
- 局部分析：系统对当前输入形成结构化解释，产出潜台词、场景判断、情绪得分与可选回应。
- 反馈转化：系统把分析结果转化为可练习、可选择、可保存的反馈对象，而不是停留在一次性展示。
- 长期沉淀：系统把练习与关系信息重新汇回关系对象，使后续分析逐步获得更稳定的上下文支撑。

6.4 关键规则

上述闭环若要真正成立，需要至少四条关键规则维持一致性。

6.4.1 规则一：所有局部分析都允许显式上下文化

当前解码服务的输入结构已经体现出这一规则：除了当前文本本身，方法层还保留了自由上下文与关系对象标识两个扩展入口。其意义在于，系统默认允许“当前文本 + 关系信息”的联合解释，而不是把上下文视为后处理补充。

这一规则带来两层约束。

- 分析入口必须允许关系对象进入主路径，否则“关系上下文优先”只能停留在文档叙述层。
- 关系上下文应以最小必要信息组织，既能帮助解释，又不把无关信息无限扩展为提示噪声。

6.4.2 规则二：输出必须结构化，而非仅返回自由文本

Pebble 的方法要求分析结果至少包含三类信息：对当前话语的解释、对风险或情绪状态的压缩判断、以及可供用户实际使用的回应候选。陪练模块同样遵循这一逻辑，即使它表现为对话界面，仍然持续产出结构化评分、即时反馈与注意点。

结构化输出的重要性在于，它决定了结果能否进入下一步。

- 若结果只有一段泛化说明，用户难以比较不同策略，也难以把某条回应保存为练习材料。
- 若结果具备固定槽位，例如潜台词、得分、方案与反馈，那么它就可以稳定进入练习本、统计与后续复盘路径。

6.4.3 规则三：练习材料是方法闭环中的中间对象

当前练习服务把解码结果与模拟陪练结果收敛为同一种练习本对象，这说明 Pebble 的方法并不把“保存”视作附属功能，而是把它视为从局部理解走向长期能力积累的关键中间环节。

在方法层，练习材料至少承担三种作用。

- 它把当前一次分析中的候选回应固定下来，避免结果随着页面关闭而消失。
- 它为用户提供跨场景比较与复盘的基础，使不同来源的应对策略可以进入同一经验空间。
- 它为后续长期洞察提供间接证据，帮助系统理解哪些策略在什么关系对象和场景下更常被采纳。

6.4.4 规则四：长期洞察以描述性上下文累积，而非一次性定性

关系服务当前维护的字段结构说明，Pebble 的长期建模中心是“关系对象”，而非对人格或状态做最终判定。关系类型、对方特点、期望结果、情境补充和生成上下文共同构成一个可被持续修订的解释条件集合。该集合在方法上的价值，在于它允许系统把局部分析结果重新放回关系语境中理解。

这一规则直接约束了长期建模的表达方式。

- 长期洞察应优先使用描述性信息和可回看的观察线索。
- 生成上下文应服务于后续解释与陪练，不应被包装成权威性诊断结论。
- 关系对象的更新应允许用户修订和重写，以保持用户主权与边界克制。

6.5 方法层的综合理解

综合来看，Pebble 的方法并非由某一个算法名称定义，而是由一组稳定的对象关系与规则关系定义。它把关系对象作为长期条件，把当前输入作为局部触发，把结构化分析作为中介，把练习记录作为沉淀载体，并把长期洞察作为下一轮分析的先验。这个方法结构能够同时解释解码、陪练、练习本与关系档案为何需要共同存在，也为下一章展开算法与执行路径提供了统一入口。

6.6 本章小结

本节读完后，读者应能画出 Pebble 的最小方法闭环（条件注入、局部分析、反馈转化、长期沉淀），并说明练习记录在闭环中的中间对象作用。

第七章 算法、流程与执行路径

7.1 本章作用

第六章已经把 Pebble 的方法对象与关键规则固定下来。本章继续回答一个更具体的问题：这些方法对象在系统中如何被串联为可执行路径，以及用户从进入页面到得到反馈、保存结果、回流上下文之间，具体经历了哪些状态转换。若说上一章处理的是“系统应如何理解问题”，这一章处理的就是“系统如何把这种理解落实为一条可走通的执行链”。

本章关注三条主路径：读心翻译路径、模拟陪练路径与稳态恢复路径。同时，还要补上一条横向的状态流，即“当前关系对象”的选择如何跨页面穿透解码与个人中心，使上下文条件能够随用户移动而保持连续。

7.2 总流程概览

从用户视角看，Pebble 当前的主流程可以压缩为一条入口分流结构：用户先从首页或个人中心进入某一功能面，再根据当前压力状态和任务目标选择不同路径。图 7.1 给出了这一结构的总览。

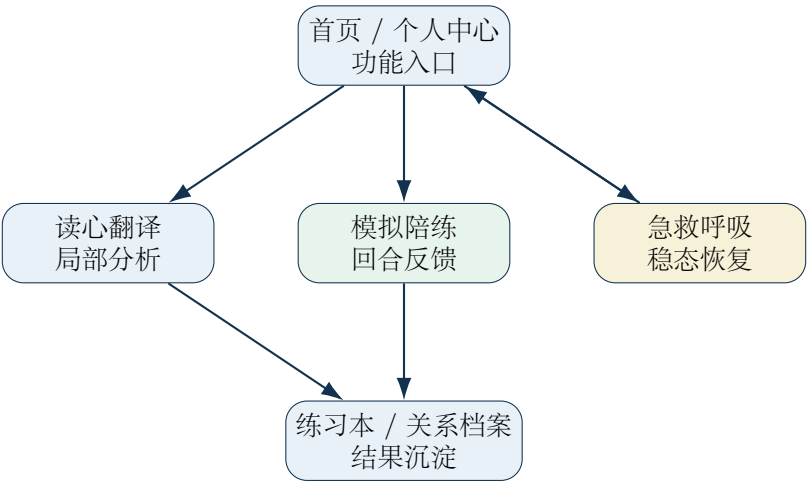


图 7.1: Pebble 的执行路径总览：三条主路径围绕同一入口展开，并在练习沉淀与关系上下文中重新汇流。

这张图表达了三个方法层判断。

- 用户入口是统一的。首页负责把功能面并列展示，个人中心负责把关系对象与功能入口重新组织在同一个用户语境下。
- 三条主路径分工不同。解码路径强调对当前话语的解释，陪练路径强调回合式练习与即时反馈，呼吸路径强调在高压时刻先恢复状态。
- 结果会重新汇流。翻译与陪练的输出都可以进入练习材料，关系对象又会影响下一轮分析，因此整个系统呈现为循环而非单次跳转。

7.3 路径一：读心翻译的执行链

（本章使用“读心翻译”作为产品功能名称，其技术实现对应第 3 章定义的“对话解码”过程。）

读心翻译路径是 Pebble 当前最直接的分析入口。其执行顺序可以分为六步。

- 用户在翻译页输入需要理解的文本。
- 状态层先校验输入是否为空，并把界面状态切换到分析中。

- 系统读取当前全局选中的关系对象标识，将其与当前文本一并发送到解码接口。
- 解码服务在后端拼接最小必要的关系上下文，并对文本执行结构化分析。
- 分析结果返回前端后，被组织为潜台词、情绪状态与三档回应建议。
- 用户可直接重试，也可把某一条候选回应存入练习本，作为后续回看材料。

这一条路径的关键在于“关系选择”与“解码请求”之间的穿透关系。前端翻译状态并不单独维护一份关系上下文，而是直接读取全局用户中心状态中的当前关系对象标识。这样一来，用户在个人中心完成关系切换后，进入翻译页即可获得上下文增强分析，无需再次重复选择。这一设计让“关系对象作为解释条件”真正进入了解码主路径。

7.4 路径二：模拟陪练的回合流程

模拟陪练路径承载的是更长的交互链。它相较于读心翻译增加了场景选择、会话状态、轮次消息与即时评分，因此更接近一个轻量的回合式执行系统。

其主流程可以概括为如下步骤。

- 页面初始化时加载场景列表，并根据 URL 参数或用户点击结果选定当前场景。
- 用户选择场景后，前端创建新会话请求，后端返回 AI 开场白与右侧分析面板的初始信息。
- 用户每发送一轮消息，前端先把该消息写入本地消息流，再进入“AI 正在响应”的中间状态。
- 后端对这一轮用户输入做情绪与边界分析，同时生成下一轮 AI 回复，并把评分、标签、即时反馈和注意点一起返回。
- 前端收到结果后，将 AI 回复附加到消息流，并把右侧面板更新为最新的训练反馈。
- 用户可继续发送下一轮，也可选择重启或结束会话。结束后，会话摘要与高频回应可进一步进入练习沉淀路径。

这条路径体现出 Pebble 在执行层的一个重要特征：系统不只给出一个静态答案，而是维持一个随轮次更新的状态机。场景、会话、消息流、右侧分析面板与结束摘要共同构成了训

练过程中的中间状态，使用户能够在练习中观察自己回应风格的变化。

7.5 路径三：稳态恢复的短流程

急救呼吸路径承担的是与前两条路径不同的职责。它不以语言分析为中心，而以状态恢复为中心，因此执行流程更短、交互目标更明确。

- 用户进入呼吸页后，系统立即开始倒计时与呼吸节奏动画。
- 页面围绕 4-7-8 周期自动切换吸气、屏息与呼气阶段，并用视觉律动同步提示当前阶段。
- 用户可随时暂停、继续或在完成后重新开始。
- 当用户状态恢复后，再回到首页、翻译页或陪练页进入后续理解与应对路径。

这一条路径的意义在于，它为整个系统提供了一条“先恢复，再分析”的执行分支。对于高压状态下的用户，立即给出语言分析并不总是最优入口；先稳定状态，再回到分析与练习链路，更符合第五章中“先理解后应对”与“边界克制”的设计要求。

7.6 横向状态流：关系对象如何跨路径保持连续

除了三条主路径之外，Pebble 还有一条横向的状态流，即当前关系对象的跨页面保持。个人中心负责加载和展示当前关系，关系选择结果会被持久化保存；翻译路径在发起解码请求时直接读取这一状态；个人中心再次打开时又会根据保存的标识重新加载关系详情。

这一横向状态流带来三点执行层收益。

- 用户无需在每个页面重复指定同一关系对象，交互成本更低。
- 关系信息进入了解码主路径，分析结果能够更稳定地体现情境差异。
- 个人中心、关系图谱与核心功能面之间形成统一的用户语境，系统不再表现为若干互不相干的工具页。

7.7 执行边界与失败处理

Pebble 当前的执行路径还包含一组必要的边界处理。它们决定系统在异常、空输入、会话失效或用户未完成前置动作时如何退回安全状态。

- 解码路径在空输入时直接返回错误状态，避免无意义请求进入分析服务。
- 陪练路径要求会话已启动且消息非空，否则会阻断发送并给出错误提示。
- 会话结束后，陪练路径不再接受继续发送，而是要求用户显式重启或重新开始。
- 呼吸路径允许暂停与重启，从而把恢复过程设计成可控节奏，而非强制性流程。

这些边界处理虽然并不直接构成产品卖点，却是执行路径成立的前提。没有这些状态约束，系统很容易在多轮交互中出现上下文漂移、空请求分析或会话错位，从而削弱整个闭环的稳定性。

7.8 本章小结

第七章说明了 Pebble 如何把第六章的方法对象组织为三条主路径与一条横向状态流。读心翻译路径负责单次局部解释，模拟陪练路径负责回合式练习与即时反馈，急救呼吸路径负责稳态恢复，而个人中心与关系选择负责把关系对象持续注入这些路径。下一章将在此基础上继续展开系统、数据与资源设计，回答这些流程在工程层面由哪些模块、接口与数据结构共同支撑。

第八章 系统、数据与资源设计

8.1 本章作用

前两章已经分别说明了执行路径与方法结构，本章进一步回答一个工程层面的问题：这些路径和方法由哪些系统层次、数据对象与运行资源共同支撑。这里讨论的重点不在于页面外观，也不在于单个接口的局部逻辑，而在于 Pebble 如何把前端状态、前端请求适配、API 路由、服务层、仓储层与数据库连接组织成一个可维护的整体。

这一章尤其需要明确系统边界。Pebble 当前既包含明显的交互层，也包含关系档案、练习沉淀、会话记录与匿名访客支持等数据层能力。如果不把这些层次区分开，系统很容易被误读为“只有几个页面 + 几个接口”的轻量前端项目；而从当前结构看，它已经具备一个小型全栈支持系统的基本骨架。

8.2 系统分层与边界

Pebble 当前可以被理解为一个六层结构。图 8.1 对这一结构做了压缩描述。

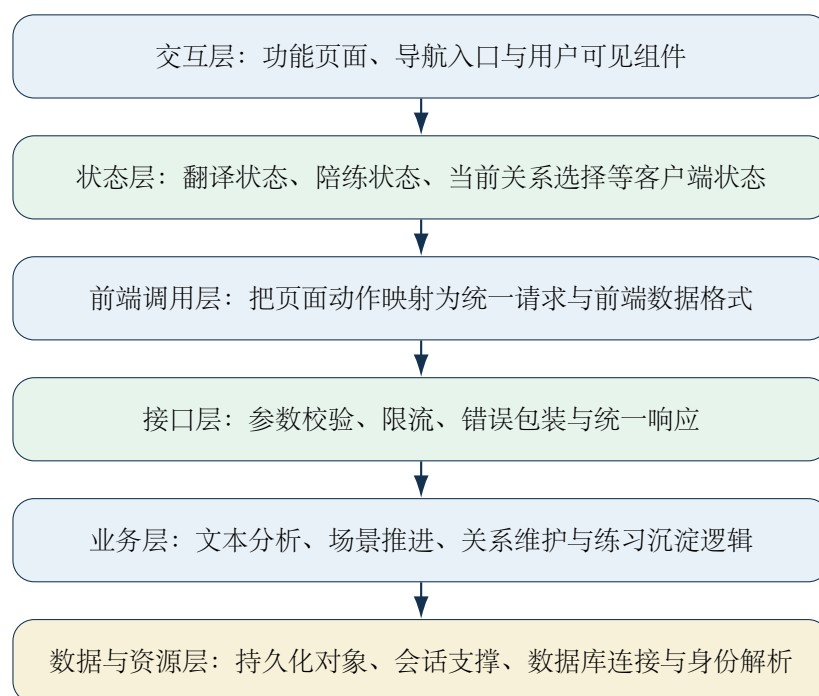


图 8.1: Pebble 当前系统分层：从页面交互到底层数据与会话资源，形成清晰的责任链。

该分层的意义在于，每一层都承担了相对清晰的责任。

- 交互层负责把产品能力组织成可感知页面，例如首页入口、翻译页、陪练场、呼吸页与个人中心。
- 状态层负责维护界面会话中的局部状态，例如翻译结果、陪练消息流与当前选中关系对象。
- 前端调用层负责把页面动作转换为统一的 HTTP 请求，并把接口响应映射回前端所需的数据形状。
- 接口层负责承接请求、做参数校验与限流，并把业务调用包装成稳定的 API 响应。
- 业务层负责形成真正的领域逻辑，例如关系上下文拼接、模拟场景推进、练习本写入与关系对象生成上下文。
- 数据与资源层负责持久化、查询、匿名会话与登录用户映射，为上层逻辑提供稳定底

座。

8.3 前端状态设计

Pebble 的前端状态层采用 Zustand 管理，当前包含四个主要 store，分别对应翻译、陪练、关系与用户中心四个功能域。它们并非彼此隔离，而是通过显式的跨 store 调用来实现状态同步。表 8.1 对这四个 store 的状态字段与职责做了汇总。

表 8.1: Pebble 前端 Zustand store 结构。

Store	状态字段	职责
useTranslatorStore	inputText, status, result, error	管理翻译页的输入、解码状态与结果展示
useDojoStore	currentScenario, messages, rightPanel, sessionId, sessionStatus	管理陪练场的场景、消息流、会话状态与右侧面板反馈
useRelationStore	nodes, selectedNodeId, isLoading, error	管理关系对象的增删改查与选中状态
useUserCenterStore	selectedRelationId, selectedRelation	管理全局当前关系选择，通过 persist 中间件持久化到 localStorage

四个 store 之间的通信关系如下：

- useTranslatorStore 在调用 decode 时，通过 useUserCenterStore.getState() 读取当前选中的关系标识，将其附加到请求中。这说明翻译模块对全局关系状态的依赖是显式的运行时调用，而非通过 React Context 或 props 传递。
- useUserCenterStore 使用 Zustand 的 persist 中间件，只持久化 selectedRelationId 到 localStorage。页面刷新后，store 会恢复该 ID，并在需要时通过 loadSelectedRelation 重新拉取关系详情。
- useDojoStore 与 useRelationStore 之间没有直接依赖，它们各自通过前端 client 层与后端通信，但共享同一套 guest session 身份机制。

状态层的设计决策有以下两点：

- 每个 store 只负责一个功能域的状态，避免了单个大 store 带来的频繁重渲染问题。
- 跨 store 通信采用显式的 `getState()` 调用，而不是通过订阅或事件总线，这使得数据流更易追踪，但也意味着 store 之间存在编译时不可见的隐式依赖。

8.4 核心模块划分

若把上述分层进一步压缩为面向报告阅读的模式视角，Pebble 当前至少包含四个核心模块。

表 8.2 对四个核心模块做了汇总。

表 8.2: Pebble 的核心模块及其对应的系统职责。

模块	主要职责	典型系统位置
解码模块	接收文本与关系上下文，返回潜台词、情绪状态与回应建议	翻译页、客户端解码状态、分析接口与文本分析逻辑
陪练模块	维护场景、会话与轮次消息，持续返回即时反馈与下一轮回复	陪练页面、回合状态、会话接口与场景推进逻辑
关系模块	维护关系档案、生成上下文与当前关系选择，并为其他模块提供解释条件	个人中心、关系选择状态、关系接口与关系对象维护逻辑
练习沉淀模块	把解码或陪练结果固化为可收藏、可归档、可回看的练习材料	练习材料接口、沉淀逻辑与长期保存对象

这四个模块之间并非松散并列关系。关系模块为解码模块提供条件，解码模块与陪练模块为练习沉淀模块提供来源，陪练模块和呼吸模块又共同承担“用户在压力情境中的支持链”不同阶段的职责。因此，Pebble 当前的结构应被理解为共享底层数据与状态语境的一组协同模块。

8.5 模块依赖与接口契约

若只从模块职责表理解系统，仍难以判断模块之间如何协作。本节进一步说明核心模块的依赖关系与服务层暴露的接口契约。

8.5.1 模块依赖关系

从代码库的导入关系可以画出以下依赖图：

- 解码模块：`translator-store` → `decode-client` → `DecodeService` → `RelationService` / `relation-repository`。这说明翻译路径在发起分析时，需要关系模块提供上下文拼接能力。
- 陪练模块：`dojo-store` → `simulator-client` → `SimulatorService` → `simulator-repository`。陪练模块相对独立，不直接依赖关系模块，但依赖同一套 `guest session` 基础设施。
- 关系模块：`relation-store` → `relation-client` → `RelationService` → `relation-repository`。关系模块是基础服务层，被解码模块调用。
- 练习沉淀模块：`translator-store` / `dojo-store` → `practice-client` → `PracticeService` → `practice-repository`。练习沉淀是跨模块的收敛点，接收来自解码和陪练两路的保存请求。

8.5.2 业务层服务接口

业务层以 `TypeScript class` 形式实现，每个服务类暴露一组类型化的公共方法。以下列出核心方法签名及其约束，见表 8.3、表 8.4 与表 8.5。

表 8.3: `DecodeService` 公共方法契约。

方法	签名	约束
<code>analyzeText</code>	<code>async (options: { text, context?, relationId?, skipPII? }) => DecodeResponse</code>	<code>text</code> 非空且不超过 5000 字； <code>relationId</code> 存在时自动拼接关系上下文；LLM 超时 10 秒则退回到启发式分析

表 8.4: SimulatorService 公共方法契约。

方法	签名	约束
getScenarios	() => ScenarioItem[]	返回固定排序的三个场景
startSession	async (scenarioId, guestSessionId?, userId?) => SimulatorResponse	scenarioId 必须在预设目录中；会话初始消息从 openings 中随机选取
processTurn	async (sessionId, message, guestSessionId?) => SimulatorResponse	会话必须处于 active 状态；用户消息存入 turns 表并返回 AI 回复与评分
restartSession	async (sessionId, guestSessionId?, userId?) => SimulatorResponse	旧会话标记为 completed，新建同场景会话
endSession	async (sessionId) => SimulatorEndResponse	计算用户 turns 的平均得分并生成总结反馈

表 8.5: RelationService 公共方法契约。

方法	签名	约束
list	async (userId) => RelationNodeDTO[]	按 position 排序返回
create	async (params: CreateRelationParams) => RelationNodeDTO	name 非空且不超过 100 字符；单个用户最多 10 个节点；自动生成 Socratic 上下文
update	async (id, userId, params) => RelationNodeDTO	鉴权：只能修改自己创建的关系
delete	async (id, userId) => void	鉴权同上
regenerateContext	async (id, userId) => RelationNodeDTO	基于现有字段重新生成 SocraticPrompt-Template

8.5.3 错误码设计

业务层统一使用 BackendError 层级，包含 8 个错误码及其对应的 HTTP 状态码：

业务层统一使用 BackendError 层级，包含 8 个错误码及其对应的 HTTP 状态码，见表 8.6。

表 8.6: BackendError 错误码映射。

错误码	HTTP 状态	触发场景
BAD_REQUEST	400	输入校验失败、参数缺失
UNAUTHORIZED	401	未登录且需要登录态的操作
FORBIDDEN	403	用户尝试操作不属于自己资源
NOT_FOUND	404	关系、会话或练习条目不存在
RATE_LIMITED	429	请求频率超过阈值
TIMEOUT	408	LLM 调用或下游服务超时
INTERNAL_ERROR	500	未预期的运行时异常
SERVICE_UNAVAILABLE	503	数据库或外部服务不可用

8.6 数据对象与持久化组织

从数据层看，Pebble 已经形成围绕用户、会话、关系与练习展开的多种持久化对象。

- `user_profiles` 用于承接登录用户的本地画像与模型偏好。
- `guest_sessions` 用于支撑匿名用户的 cookie 会话，使未登录用户也能进入部分支持链路。
- `relation_nodes` 用于存放关系档案，是关系上下文与长期洞察的核心承载对象。
- `analysis_logs` 用于记录分析元数据，为后续统计与复盘提供基础。
- `simulation_sessions` 与 `simulation_turns` 共同维护陪练会话及其轮次消息。
- `practice_entries` 用于收敛来自解码与陪练的练习材料，形成跨路径沉淀。
- `panic_sessions` 与 `user_activity_stats` 用于记录稳态恢复和用户日常活跃信息。

这些对象可以进一步分成三组。

- 用户与身份组: `user_profiles`、`guest_sessions`。
- 互动过程组: `analysis_logs`、`simulation_sessions`、`simulation_turns`、`panic_sessions`。
- 长期记忆组: `relation_nodes`、`practice_entries`、`user_activity_stats`。

这种分组方式与前文的方法结构是一致的。互动过程组承载“当前输入与当前会话”，长期记忆组承载“练习记录与关系对象”，身份组则决定这些对象由匿名用户还是登录用户持有。

8.7 数据库实体关系与 JSONB 结构

8.7.1 实体关系

从数据库 schema 可以画出以下核心关系：

- `user_profiles` (1) → (N) `relation_nodes`: 一个用户可拥有多个关系对象，通过 `userId` 外键关联，级联删除。
- `user_profiles` (1) → (N) `practice_entries`: 一个用户可保存多条练习记录。
- `guest_sessions` (1) → (N) `practice_entries`: 匿名访客也可保存练习记录，`guestSessionId` 与 `userId` 互斥（一条记录只绑定一种身份）。
- `simulation_sessions` (1) → (N) `simulation_turns`: 一个陪练会话包含多条轮次消息，通过 `sessionId` 外键关联，级联删除。
- `user_profiles` (1) → (N) `simulation_sessions`: 登录用户创建的陪练会话绑定到用户；匿名访客创建的则绑定到 `guest session`。

`analysis_logs`、`panic_sessions`、`user_activity_stats` 目前为独立表，未与其他业务表建立外键关系，分别通过 `userId` 或 `guestSessionId` 进行弱关联。

8.7.2 JSONB 字段结构

数据库中有三个表使用 JSONB 存储结构化快照，其内部 schema 见表 8.7。

表 8.7: JSONB 字段结构定义。

表名	JSONB 字段	内部结构
practice_entries	content_jsonb	originalText : 原始输入文本; analysis : { attackType, scenario, subtext, emotionScore, neutralityScore }; replyOptions : { id, label, content, tone }
simulation_turns	analysis_jsonb	score : 情绪得分; label : 等级标签; feedback : 即时反馈文本
simulation_sessions	history_snapshot	totalTurns : 轮次总数; averageScore : 平均 得分

使用 JSONB 而非独立列的原因是：这些字段的内部结构可能随产品迭代而调整，JSONB 允许在不解耦 schema 的情况下扩展新字段。但其代价是失去了数据库层面的类型约束，需要在前端 DTO 层和服务层做额外的运行时校验。

8.8 设计系统架构

Pebble 的设计系统并非仅是一套视觉规范，而是通过 TypeScript 令牌文件、Tailwind CSS 配置与可复用组件三层共同实现的工程化体系。

8.8.1 设计令牌

所有视觉常量集中在 `lib/design-system/tokens.ts` 中定义，分为五组：

- 颜色令牌: `pebbleColors` (`rock`, `healing`, `background`, `accent`, `warn`, `text`, `shelter`, `storm`, `insight`)
- 字体令牌: `pebbleFonts` (`serif`, `sans`, `display`)
- 圆角令牌: `pebbleRadii` (有机鹅卵石形状 `pebble` 及其三个变体 `pebble1/2/3`，以及标

准 pill)

- 阴影令牌: pebbleShadows (glass, soft, glow)
- 动效时长: pebbleMotionDurations (brisk 120ms, gentle 200ms, reveal 320ms, ambient 600ms)

这些令牌同时通过 CSS 变量注入到 `globals.css` 中, 使组件既可以消费 TypeScript 常量, 也可以直接引用 CSS 变量。

8.8.2 组件层级

设计系统组件按职责分为三层:

- 基础层 (atoms): PebbleButton、GlassCard、StatusPill、PebbleInputShell。它们只依赖设计令牌, 不依赖业务状态。
- 布局层 (layouts): PageShell、MainHeader、CenteredStageLayout。它们负责页面级容器和导航结构。
- 业务组件 (molecules/organisms): BreathingOrb、DecoderDemo、ScenarioCard、CoachingPanel。它们组合基础层组件, 并接入业务状态。

这种分层使得基础层组件可以被独立测试和复用, 而业务组件则专注于状态与交互逻辑。

8.8.3 呼吸状态机

急救呼吸页的核心逻辑被提取为纯函数状态机 `lib/breathing/breathing-machine.ts`, 与 React 完全解耦。它只暴露三个导出:

- `BreathingPhase` 类型: `'idle' | 'inhale' | 'hold' | 'exhale'`
- `BREATHING_CYCLE` 常量: 定义 4-7-8 三阶段时长 (毫秒)
- `nextPhase(phase)` 函数: 按 `inhale → hold → exhale → inhale` 循环推进

UI 层通过 `useBreathing` Hook 消费该状态机, 负责把阶段映射到动画关键帧与文案提示。这种分离使呼吸逻辑可以在无 DOM 环境中进行单元测试。

8.9 匿名会话与登录用户的双轨支撑

Pebble 当前的一个重要系统特征，是同时支持匿名访客与登录用户两种路径。匿名访客主要通过 `guest session` 进入解码、陪练与练习沉淀路径；关系档案相关能力则更多依赖登录用户及其本地用户画像。

这一双轨结构的工程含义在于，系统没有把所有功能都强制绑定到登录门槛上，而是采用了更细粒度的能力划分。

- 匿名路径通过 `cookie` 中的 `guest session token` 建立最小身份，使访客也能获得限流、记录与部分持久化能力。
- 登录路径通过 Supabase 用户解析到 `user_profiles`，再把业务数据绑定到本地用户标识，而非直接把认证用户 ID 散落到全部业务逻辑中。
- 关系模块采用显式鉴权策略。若数据库不可用、用户未登录或本地画像解析失败，系统会在接口层以区分明确的错误类型返回，而不是把所有失败都混成统一异常。

这一设计提升了系统的可迁移性与边界清晰度。匿名访客可以先体验核心功能，登录用户则可以逐步进入关系档案与长期个性化路径，两者在数据层又通过明确的 `guestSessionId` 与 `userId` 关联保持可区分性。

8.10 数据流实例：从用户输入到持久化

为了把上述分层、模块与数据对象之间的关系具体化，本节以“读心翻译”路径为例，追踪一条用户输入从页面到数据库的完整数据流。

1. 用户输入：用户在翻译页输入文本，触发 `useTranslatorStore.decode()`。
2. 状态层校验：store 检查输入非空且未处于 `analyzing` 状态，将 `status` 设为 `'analyzing'`。
3. 跨 store 读取：`decode` 方法通过 `useUserCenterStore.getState().selectedRelationId` 获取当前关系标识。
4. 前端调用层：`decode-client.ts` 将 `{ text, relationId }` 序列化为 JSON，通过 `fetch('/api/decode')` 发送 POST 请求，并设置 15 秒超时。
5. 接口层：Next.js Route Handler 接收请求，校验 `Content-Type` 与请求体结构，提取

guest session cookie。

6. 业务层: `DecodeService.analyzeText()` 被调用。它依次执行:

- 输入长度校验 (≤ 5000 字符)
- PII 脱敏 (调用 `redactSensitiveText`)
- 关系上下文拼接 (若 `relationId` 存在, 查询 `relation_nodes` 表并拼接字段)
- LLM 分析 (带 10 秒超时)
- 若 LLM 失败, 退回到启发式关键词匹配分析
- 生成三档回复建议
- 持久化分析元数据到 `analysis_logs` 表

7. 响应映射: 业务层返回 `DecodeResponse`, 接口层包装为统一响应体返回前端。

8. 前端状态更新: `decode-client.ts` 解析响应并映射为前端 `DecodeResponse` 类型, `store` 将 `result` 与 `status` 更新为 `'result'`。

9. UI 渲染: 翻译页根据 `result` 渲染表面语义、潜台词、情绪状态与三档回复建议。

10. 可选保存: 用户点击“存入练习本”时, 触发 `practice-client.ts` 调用 `POST /api/practice`, `PracticeService` 将结果写入 `practice_entries` 表的 `content_jsonb` 字段。

这条路径展示了 Pebble 六层架构的完整协作: 状态层管理交互状态, 调用层统一网络请求, 接口层做边界校验, 业务层执行业务逻辑与降级策略, 数据层完成持久化。每一层只与相邻层通信, 不跨层直接调用。

8.11 资源与退化策略

系统设计不仅要说明“正常情况下如何工作”，还要说明关键资源缺失时如何退化。Pebble 当前已经可以观察到至少三类资源边界。

- 数据库资源：若 `DATABASE_URL` 不可用，数据库连接会退化为不可用状态，部分服务则转入降级模式。
- 会话资源：若用户没有匿名会话 token，则系统在需要时创建新的 guest session，保证匿名路径仍可运行。
- 模型与分析资源：解码服务在分析超时或失败时会退回启发式分析，陪练模块在数据库不可用时也具备内存回退路径。

这些退化策略具有两层价值。

- 它们提高了系统在开发与轻量部署环境中的可运行性。
- 它们为后续章节中的“项目进度”与“风险边界”提供了更清晰的分类依据，使读者能够区分哪些能力依赖完整后端资源，哪些能力在资源不完整时仍能以弱化方式继续工作。

8.12 本章小结

本节读完后，读者应能描述 Pebble 的六层架构中每一层的职责边界，并指出四个核心模块之间的协作关系。本节读完后，读者应能描述 Pebble 的六层架构中每一层的职责边界；画出前端四个 Zustand store 的状态字段与跨 store 通信关系；说明业务层核心服务的方法签名与错误码设计；解释数据库表之间的实体关系与 JSONB 字段的使用理由；并追踪一条用户输入从页面到数据库的完整数据流路径。

下一章将在此基础上继续进入示例、结果与验证，讨论这些结构如何在实际交互中体现为可观察行为与可验证结果。本章补充了此前缺失的前端状态设计、服务接口契约、数据库实体关系、JSONB 结构定义、设计系统架构与完整数据流追踪。下一章将在此基础上继续进入示例、结果与验证，讨论这些结构如何在实际交互中体现为可观察行为与可验证结果。

第九章 示例、结果与验证

9.1 本章作用

前八章已经给出了 Pebble 的问题域、方法结构、执行路径与系统骨架。本章继续回答一个更直接的问题：如果把这些设计放回真实交互中，当前到底能观察到哪些结果，又有哪些结论已经具备验证证据。这里的重点不是夸大系统效果，而是用代表性示例、现有页面行为、接口返回结构与测试证据，区分“当前已观察事实”与“仍需后续验证的设计意图”。

9.2 代表性示例

Pebble 当前最有代表性的示例来自三条主路径：读心翻译、模拟陪练与急救呼吸。它们分别对应理解、练习与稳态恢复三个阶段，因此能够共同展示系统闭环的基本轮廓。

9.2.1 示例一：读心翻译的结构化结果

在读心翻译路径中，页面默认示例与当前组件结构都表明，系统会把一次文本输入压缩为三类核心结果。

- 表面语义：对原始输入做较为中性的重述。
- 潜台词：对对方话语背后的压力结构或情绪意图做压缩说明。
- 回复建议：给出三档候选回应，并允许复制或存入练习本。

这一结果结构与前文方法章是吻合的。当前页面中，用户在得到结果后可以继续分析、复制建议，或者把选中的方案写入练习本。由此，翻译路径展示出的不是单一自由文本回答，而是一种可被继续操作的结构化分析结果，包含表面语义、潜台词与三档回复

建议。

9.2.2 示例二：模拟陪练的回合反馈

在模拟陪练路径中，系统返回的不是静态分析页，而是一组随回合推进而更新的反馈对象。系统返回一组随回合推进而更新的反馈对象，包括情绪得分、标签、即时反馈与注意点。当前结构中，右侧面板至少包含情绪得分、标签、即时反馈与注意点。用户每发送一轮消息，这些对象都会随着后端返回结果而更新。

这意味着陪练结果可从两个层面理解。

- 它给出局部结果，即当前这一轮回应在中性度或边界维护上的即时反馈。
- 它也构成过程结果，因为多轮反馈会共同形成一次练习会话的轨迹。

与读心翻译相比，陪练路径更像对方法闭环的动态展开。系统不只判断一句话，也持续评估用户在压力语境中的回应方式是否保持克制、是否滑入解释或对抗。

9.2.3 示例三：急救呼吸的状态恢复反馈

急救呼吸路径的结果并不表现为文本建议，而表现为一个可观察的同步控制过程。页面当前可直接观察到 4-7-8 节奏、阶段文字同步与 1:59 倒计时。这里的结果属于状态型结果，即系统通过持续节奏引导帮助用户进入更稳定的呼吸与注意力节奏。

这一路径与前两条路径共同说明，Pebble 的结果面并不统一收敛为“文本答案”。在不同场景下，系统输出可以是结构化分析、回合反馈，也可以是恢复节奏本身。正因为存在这种差异，后文的验证也必须按路径分别处理，而不能用单一指标概括全系统。

9.3 当前已观察结果

若严格按照当前工作区中可直接观察到的事实来判断，Pebble 至少已经具备表 9.1 所示的结果面。

表 9.1: Pebble 当前可直接观察到的结果类型及证据边界。

路径	当前可观察结果	证据边界
读心翻译	页面可展示表面语义、潜台词、情绪状态与三档回复建议，并支持复制与存入练习本	可确认结果结构已存在；不可仅据此宣称语义质量已充分验证
模拟陪练	页面可展示场景对话、AI 回复、情绪得分、即时反馈与注意点	可确认回合反馈链已建立；不可仅据此宣称训练效果已完成长期验证
急救呼吸	页面可展示 4-7-8 节奏、阶段文字、倒计时与暂停/重启交互	可确认恢复流程可运行；不可仅据此推断临床级稳定效果
练习沉淀	解码与陪练结果都可以进入练习材料对象	可确认沉淀入口存在；不可仅据此断言长期复盘价值已被量化
关系上下文	当前关系对象可穿透到翻译分析路径，形成上下文增强入口	可确认上下文注入链已存在；不可仅据此断言所有关系适配都足够精准

上述结果的共同特点是，它们都属于“结构与行为已可观察”的层面。也就是说，当前可以确认系统已经形成若干稳定的输入输出形状、交互动作与状态流；但对这些结果是否已经达到高质量、强泛化或长期有效的程度，仍需更严格的验证证据支持。

9.4 验证证据

当前与 Pebble 相关的验证证据主要来自三类来源。

- 页面与接口层的直接观察证据：当前工作区中的页面组件、状态管理与接口返回结构，能够直接证明结果形状和执行路径是否存在。
- 测试与前端断言证据：当前仓库已包含针对翻译页、分析卡片、解码按钮、情绪状态条、回复建议与相关交互与状态切换已经被显式检查。前端测试矩阵已覆盖翻译页、分析卡片、解码按钮、情绪状态条、回复建议与相关交互，具体覆盖范围与用例数量详见第 9 章”测试覆盖”小节。
- 任务书与测试计划证据：当前文档中已经给出了首页、翻译器、陪练场、急救呼吸与关系系统的验收项和测试步骤，为后续系统级验证提供了目标框架。已完成的执行清单与测试用例详见第 9 章”技术实现状态”与”测试覆盖”小节。

这三类证据的验证力度并不相同。

- 直接观察证据最适合证明“是否存在某条链路或某种结果结构”。
- 前端测试最适合证明“在给定状态下，界面与局部交互是否按预期工作”。
- 验收方案最适合证明“系统原本想达到什么验收标准”，但它本身不等于已经执行完毕的结果。

因此，当前最稳妥的验证结论应停留在以下层次。

- 可以确认 Pebble 的三条主路径都已经具备可观察的结果输出。
- 可以确认部分关键前端交互和显示结构已经进入测试覆盖。
- 可以确认系统已经存在较明确的验收框架与关系系统专项测试计划。

9.5 尚未完成的验证边界

本章必须同样明确哪些内容仍不能被夸大。

- 当前并没有充分证据证明解码结果在广泛语境中都具有稳定高质量。
- 当前并没有充分证据证明模拟陪练已经形成可量化的长期训练收益。
- 当前并没有充分证据证明急救呼吸路径在临床或准临床场景中达到可外推的恢复效果。
- 当前也没有充分证据证明关系上下文增强已经在大量不同关系对象上得到系统性评估。

换言之，Pebble 当前最强的证据类型是“系统链路已存在并可运行”，而不是“长期效果已被严格量化”。这一边界需要在后续章节与最终交付中持续保持，否则报告很容易从工程事实滑向未经验证的效果宣称。

9.6 本章小结

第九章给出的结论是明确的：Pebble 当前已经形成可观察的分析结果、回合反馈、恢复流程与练习沉淀入口。系统效果层面的更强结论仍需后续验证。具体实现状态、测试覆盖与待完善项详见第 9 章。

第十章 项目进度

10.1 本章作用

本章集中记录 Pebble 在当前代码库中的真实实现状态、已完成能力、技术落地细节与待完善项。它与前八章的区别在于：前八章主要围绕设计意图与系统结构展开论述，本章则把论述锚定在代码库中可直接观察到的文件、接口、测试与数据对象上。

本章的事实来源是当前工作区中的源代码、数据库迁移记录、测试文件与 OpenSpec 变更归档。所有列出的能力都以”已可运行”或”已可验证”为前提，不超前宣称尚未实现的特性。

10.2 已完成的能力面

10.2.1 前端交互层

当前代码库中已具备以下可直接运行的前端页面与组件：

- 首页：包含 Hero 主视觉、统计卡片、功能入口、读心翻译演示、模拟陪练场景卡片、急救呼吸鹅卵石动画与滚动触发动效。
- 读心翻译页：支持文本输入、情绪状态条展示、解码请求触发、分析结果展示（表面语义、潜台词、情绪得分）与三档回复建议（A/B/C）。
- 模拟陪练场页：支持场景选择（职场越界、亲密关系、社交应对）、回合式对话、右侧实时分析面板（情绪得分、标签、即时反馈、注意点）与会话结束总结。
- 急救呼吸页：支持 4-7-8 呼吸节奏引导、倒计时、暂停/重置、沉浸背景动画与波纹扩散效果。

- 个人中心与关系管理：支持关系对象创建、编辑、列表展示，以及当前关系选择对翻译分析路径的上下文穿透。
- 设计系统组件库：包含玻璃拟态卡片（GlassCard）、鹅卵石按钮（PebbleButton）、状态胶囊（StatusPill）、呼吸圆（BreathingOrb）、页面壳（PageShell）等 9 个以上可复用组件。

10.2.2 后端服务层

后端采用 Next.js Route Handlers 作为 BFF 层，当前已实现 9 个 API 端点：

表 10.1 对当前 API 端点做了汇总。

表 10.1: Pebble 当前已实现的 API 路由端点。

端点	职责	状态
POST /api/decode	接收文本与关系上下文，返回结构化分析结果	已实现
POST /api/simulator	支持 start/turn/restart 三种会话动作	已实现
POST /api/simulator/[sessionId]/end	返回会话结束总结	已实现
GET /api/scenarios	返回固定排序的场景目录	已实现
POST /api/practice	保存解码或陪练结果到练习本	已实现
PATCH /api/practice/[practiceId]	更新收藏、归档状态	已实现
DELETE /api/practice/[practiceId]	删除练习条目	已实现
POST /api/relations	创建关系对象	已实现
GET /api/relations/[id]	读取关系详情	已实现

此外，后端还包含以下策略层能力：

- PII 脱敏：在调用 LLM 前对电话、邮箱、姓名等敏感信息执行自动替换。
- 限流与超时：按 session 和 IP 双重约束限流；LLM 调用设置硬超时并具备降级路径。

- 确定性 fallback：当 LLM 供应商不可用时，后端退回基于规则的本地启发式分析，保证基础功能不中断。
- 匿名会话管理：通过 `guest_session_id` cookie 为未登录用户提供最小身份与部分持久化能力。

10.2.3 数据持久化层

数据库采用 PostgreSQL（通过 Drizzle ORM 管理），当前已执行 2 次迁移，包含 8 张核心数据表：

表 10.2 对当前数据库表做了汇总。

表 10.2: Pebble 当前数据库表及其职责。

表名	职责	数据分组
<code>user_profiles</code>	登录用户的本地画像与模型偏好	身份组
<code>guest_sessions</code>	匿名访客的 cookie 会话绑定	身份组
<code>relation_nodes</code>	关系档案、生成上下文与情境补充	长期记忆组
<code>practice_entries</code>	从解码与陪练收敛的练习材料	长期记忆组
<code>analysis_logs</code>	分析元数据（不含原始敏感文本）	互动过程组
<code>simulation_sessions</code>	陪练会话摘要与状态	互动过程组
<code>simulation_turns</code>	陪练轮次消息快照	互动过程组
<code>panic_sessions</code>	急救呼吸使用记录（待接入运行时）	互动过程组
<code>user_activity_stats</code>	用户日常活跃聚合统计	长期记忆组

10.2.4 测试覆盖

当前测试体系分为前端测试与后端测试两层：

- 前端测试：50 个测试文件，覆盖首页、翻译页、陪练场、呼吸页、个人中心、设计系统组件与自定义 Hook，累计约 448 个测试用例。
- 后端测试：6 个测试文件，覆盖 schema、contracts、错误处理、会话管理、解码服务与路由适配器。

测试框架采用 Vitest，前端测试在 jsdom 环境下运行，后端测试在 Node 环境下运行。

10.3 技术实现状态

10.3.1 设计系统落地

设计系统已通过 Tailwind CSS 配置、CSS 变量与 TypeScript 令牌文件三重方式落地：

- 颜色令牌：`pebble-rock`、`pebble-healing`、`pebble-bg`、`pebble-accent`、`pebble-warn` 已定义并在组件中消费。
- 字体系统：Noto Serif SC（标题）、Noto Sans SC（正文）、Public Sans（英文显示）已配置。
- 圆角系统：有机鹅卵石形状(`border-radius: 60% 40% 70% 30% / 40% 50% 60% 40%`) 及其变体已实现。
- 动画系统：呼吸动画（19 秒周期）、浮动背景动画（12 秒周期）与页面过渡动画均已通过 CSS keyframes 或 Framer Motion 实现。

10.3.2 OpenSpec 变更归档

项目已通过 OpenSpec 工作流完成 4 个变更的归档：

- 建立 Pebble 设计系统（TB-005）
- 构建 Pebble 首页（TB-001）
- 构建 Pebble 读心翻译器（TB-002）
- 后端 BFF 与接口技术预研（TB-006，部分完成）

另有 2 个活跃变更尚未归档：首页 UI 精细化、翻译器 UI 精细化、陪练场构建、急救呼吸构建。

10.4 当前阻塞与待完善项

10.4.1 后端 BFF 剩余工作

TB-006 任务书的后端 BFF 层虽已具备主要路由骨架，但以下子任务仍处于待完成状态：

- 部分后端仓库层测试尚未补齐（simulation-repository、session-repository 的测试覆盖较薄）。
- OpenAPI 文档（docs/backtend/openapi.yaml）需要与当前实际路由保持一致并重新校验。
- 多实例部署时的 Redis 级限流尚未接入，当前限流为单机内存实现。

10.4.2 移动端与可访问性

- 呼吸页的移动端人工验收尚未完成（TB-004 子任务 6）。
- 部分页面的 prefers-reduced-motion 降级策略已配置，但尚未经过系统性测试。

10.4.3 功能缺口

- panic_sessions 表已创建迁移，但尚未在运行时接入急救呼吸流程的自动记录。
- 用户活跃度统计（user_activity_stats）表已定义，但数据写入逻辑尚未完整实现。
- 关系档案的”长期洞察”与”策略适配”能力处于数据层就绪、分析层待建状态。

10.5 里程碑回顾

表 10.3 对当前主要里程碑做了汇总。

表 10.3: Pebble 当前已实现的主要里程碑。

时间	里程碑	证据
2026-03-19	设计系统完成 (TB-005)	颜色/字体/圆角/动画令牌落地
2026-03-21	首页完成 (TB-001)	页面、测试、导航与动效可运行
2026-03-22	读心翻译器完成 (TB-002)	解码链路、回复建议、practice 入口可用
2026-03-23	急救呼吸完成 (TB-004)	4-7-8 节奏、倒计时、动画通过测试
2026-03-24	后端 BFF 骨架完成 (TB-006 部分)	API 路由、DTO、仓库层、策略层已落地
2026-03-30	关系系统与用户中心集成	关系 CRUD、上下文穿透、Supabase Auth 绑定

10.6 本章小结

本节读完后，读者应能列举 Pebble 当前已实现的前端页面、后端端点、数据库表与测试覆盖范围，并指出至少两项尚未完成的阻塞或待完善项。

第十一章 风险、局限与下一步交付

11.1 本章作用

本章基于前九章的论述，把 Pebble 当前面临的主要风险、已知局限、交付边界与后续推进建议集中呈现。它旨在让读者在理解系统能力的同时，也清楚系统尚未跨越的边界，从而避免把”设计意图”误读为”已保证能力”。

11.2 主要风险

11.2.1 技术风险

表 11.1 对上述技术风险做了汇总。

表 11.1: Pebble 当前的主要技术风险及其缓解状态。

风险	描述	影响	缓解措施
LLM 供应商质量波动	解码与陪练的核心分析依赖大语言模型，供应商响应质量、延迟与可用性均存在不可控波动	高	已部署确定性 fallback 机制，可在供应商不可用时退回本地启发式分析
情绪敏感内容的安全边界	用户输入可能包含 crisis-like 文本，系统若处理不当可能引发伦理或法律问题	高	PII 脱敏、安全降级响应与明确的产品边界声明（非医疗诊断）
单机限流的扩展瓶颈	当前限流基于内存实现，多实例部署时无法共享限流状态	中	架构已预留 Redis 接入点，待多实例需求明确后迁移
数据库连接在开发环境的退化	Neon/PostgreSQL 连接在本地开发或网络不稳定时可能失效	中	核心接口（如解码）已具备 fallback，但持久化功能会降级

11.2.2 产品风险

- 效果验证不足：当前可以确认系统链路与结果结构已存在，但没有充分证据证明解码结果在广泛语境中都具有稳定高质量，也没有证据表明模拟陪练已形成可量化的长期训练收益。
- 用户留存依赖关系档案：关系档案是 Pebble 差异化价值的核心来源，但若用户在初期未建立足够的关系对象，系统可能退化为与通用情绪分析工具差异不大的产品。
- 匿名用户数据迁移摩擦：当前匿名访客可通过 guest session 体验核心功能，但登录后的数据合并逻辑尚待完善，可能成为用户从” 体验” 到” 留存” 的转换障碍。

11.2.3 合规与伦理风险

- Pebble 明确声明不提供心理诊断、医学治疗或人格定性，但系统输出的分析结果仍可能被用户误读为专业判断。产品层面需要在关键输出位置持续强化”建议仅供参考”的边界提示。
- 关系档案包含用户与特定关系对象的互动历史与情境补充，属于高度敏感的个人数据。数据加密、访问控制与用户删除权需要持续审计。

11.3 已知局限

- 多语言支持当前仅覆盖中文，英文界面与多语言切换尚未实现。
- 急救呼吸路径的 `panic_sessions` 记录功能已建表但尚未接入运行时，无法自动追踪用户的使用频率与恢复时长。
- 关系档案的”长期洞察”与”策略适配”仍处于数据层就绪、分析层待建状态：系统可以存储关系对象，但尚未基于多轮互动自动生成可回看的策略建议或趋势分析。
- 前端测试目前以单元测试和组件测试为主，缺乏端到端（E2E）测试覆盖完整用户旅程。
- 后端部分仓库层（`simulation-repository`、`session-repository`）的测试覆盖较薄。

11.4 交付边界

本报告所描述的 Pebble 系统，当前可稳定交付的范围包括：

- 首页、读心翻译页、模拟陪练场页、急救呼吸页与个人中心的前端交互实现。
- 解码、陪练、练习本、关系管理、场景目录的 API 端点与后端服务层。
- 基于 Drizzle ORM + PostgreSQL 的数据持久化与 2 次数据库迁移。
- 设计系统组件库与 Tailwind CSS 令牌系统。
- 前端 50 个测试文件与后端 6 个测试文件构成的回归测试矩阵。

当前不在交付范围内、但已在设计意图中规划的包括：

- 基于多轮互动的自动策略洞察生成。
- 跨设备数据同步与离线优先架构。
- 英文界面与多语言支持。
- 端到端自动化测试覆盖。
- 生产环境的多实例限流与 Redis 缓存层。

11.5 下一步建议

基于当前状态与风险排序，后续推进建议按以下优先级展开：

1. 补齐后端薄弱测试：优先为 `simulation-repository` 与 `session-repository` 补充单元测试，提升后端回归信心。
2. 同步 OpenAPI 文档：将 `docs/backtend/openapi.yaml` 与当前实际路由、DTO 和错误码重新对齐，并加入 CI 校验。
3. 完成移动端验收：对呼吸页与陪练场的移动端体验进行人工走查，补齐响应式缺口。
4. 接入 `panic_sessions` 运行时：在急救呼吸流程中自动记录使用时长、完成状态与技巧类型，为后续健康洞察提供数据基础。
5. 启动长期洞察原型：在关系档案模块中试点“基于最近 N 次分析的回复策略趋势”，验证用户是否接受系统生成的策略建议。
6. 评估 E2E 测试引入：在核心用户旅程（首页 → 翻译 → 保存 practice → 查看个人中心）上引入 Playwright 或等价的端到端测试。

11.6 本章小结

本节读完后，读者应能列举 Pebble 当前面临的至少两项技术风险与两项产品风险，并说明下一步建议的优先级排序依据。